FIELD NOTES · CLAUDE PLATFORM

Running an AI SRE on the Claude platform

How we built the agent that investigates our production incidents using the Claude Platform.

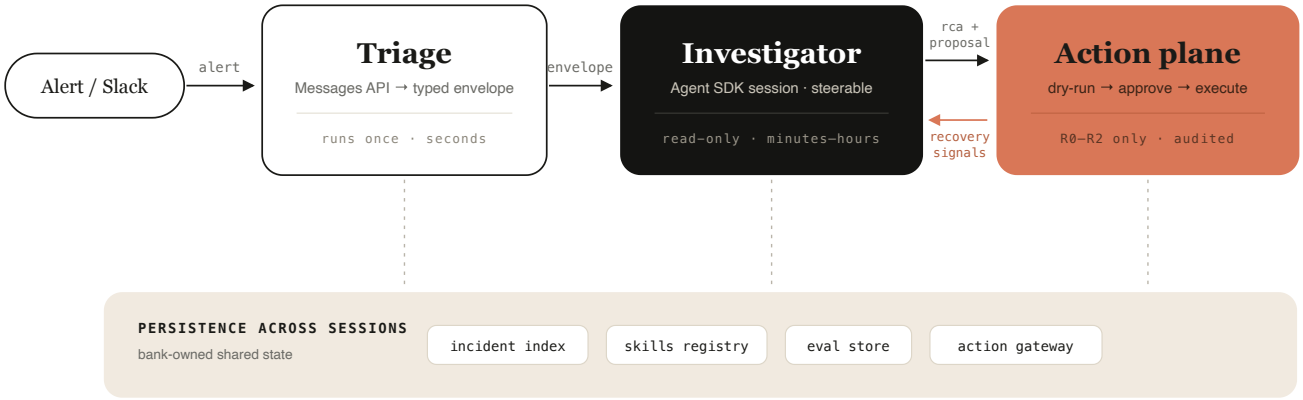
Ivaylo Bahtchevanov · July 2026

Incident response at Nubank is largely automated. Bots open the incident, propose severity, create the Slack war room, page responders, file tickets, and schedule the review. The step that still depended on a senior engineer was investigation: using judgment and experience to turn the metrics, logs, traces, deploy history, and past incidents into a credible root-cause hypothesis, often at 3 a.m.

In 2025 we shipped a read-only investigation agent, internally called Ada, on a custom-built harness, because nothing hosted at the time had the durable, steerable sessions an investigation needs. Ada worked well enough to expose the problems that matter at the next stage. It could not search past incidents, could not be redirected mid-investigation, could not act on its conclusions, ran on infrastructure that consumed most of a two-person team, and had no reliable way to measure whether its diagnoses were correct.

This post describes the v2 design. Structured outputs type every artifact another system parses, the Agent SDK runs the investigation loop, MCP carries the operational reads, code in the middle keeps raw telemetry out of the model's context, and Skills hold the runbooks service teams own. Two constraints shaped the design more than any feature choice: where session state can live, and which actions exist for the agent at all. Anthropic's cookbooks include incident-response agents on both runtimes and cover the basic mechanics; the difference here is everything a production system has to implement before the agent touches a real incident.

The architecture at a glance



Engineers only see and interact with one agent. These four components live across sessions.

An alert acts as a typed envelope on the Messages API, an Agent SDK session investigates it, and any proposed action crosses a separate gated plane. Four shared components persist across sessions.

An alert enters through a fast path on the Messages API, which classifies it and emits a typed incident envelope. The envelope starts an Agent SDK session inside our infrastructure. The session loads the relevant Skills, gathers evidence through MCP read tools, clears a grading pass, and posts a structured diagnosis into the incident's Slack thread, where the on-call engineer can label it or redirect it. If the diagnosis warrants action, the proposal goes to a separate action plane, where a human approves a dry run before anything executes. Four components sit under all of this and outlive any single session: the incident index, the Skills registry, the eval store, and the action gateway.

We split the design by whether the work can be bounded in advance. Triage, judge scoring, and the grader each have clearly defined input and output, so they run as single Messages API calls with structured outputs and no session state. The investigation cannot be bounded in the same way so it runs as an Agent SDK session, with state held as files on our infrastructure. Runbooks are Skills, maintained in the owning team's repo. Writes go through the action gateway as typed proposals.

Triage runs on the Messages API

A single API call classifies the alert, proposes a severity, and normalizes the payload into the incident envelope. The envelope starts a session, the judge's score files into the eval store, and the grader's verdict decides whether a diagnosis posts; the first reader of each is software, so a response that almost parses is a failure. Structured outputs remove that failure class at the API. The envelope schema goes in `output_config.format` and the response is guaranteed to validate against it:

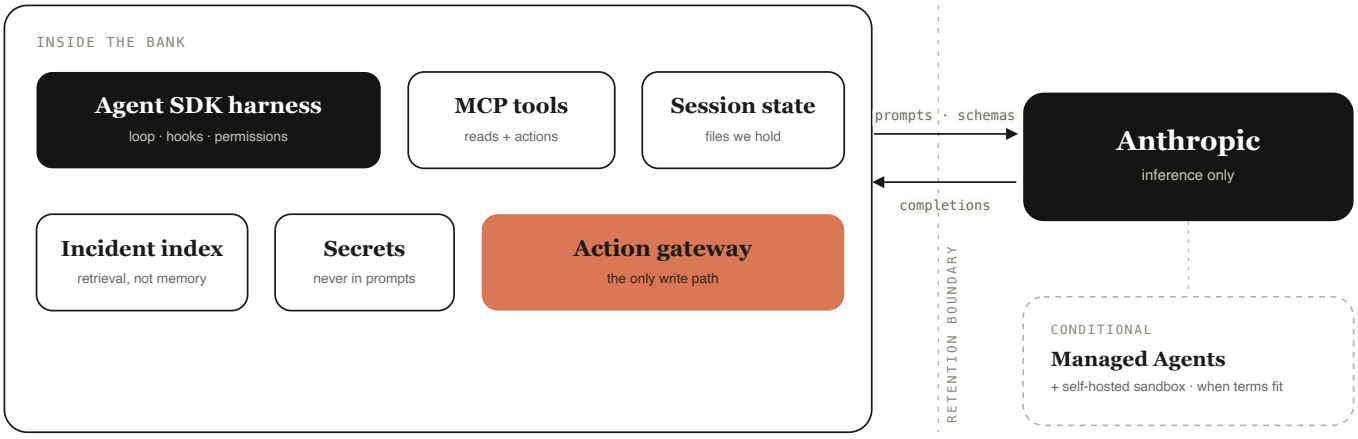
```
from anthropic import Anthropic

client = Anthropic()

envelope = client.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=1024,
    messages=[{"role": "user", "content": alert_payload}],
    output_config={
        "format": {
            "type": "json_schema",
            "schema": {
                "type": "object",
                "properties": {
                    "service": {"type": "string"},
                    "severity": {"type": "string", "enum": ["sev1", "sev2", "sev3"]
                    "error_signature": {"type": "string"},
                    "affected_components": {"type": "array", "items": {"type": "str
                    "window_start": {"type": "string"},
                },
                "required": ["service", "severity", "error_signature",
                             "affected_components", "window_start"],
                "additionalProperties": False,
            },
        }
    },
)
```

The same pattern covers every downstream artifact: the diagnosis record, the action proposal, the Slack feedback record, and eval results. In the Python SDK, `client.messages.parse()` along with the Pydantic model give the same guarantee with typed objects instead of raw JSON. Three operational details matter in practice.

The first request with a new schema pays a grammar-compilation cost; the compiled grammar is then cached for 24 hours from last use. The schema guarantee has two documented exceptions, a safety refusal and hitting `max_tokens`, so we check `stop_reason` before trusting a parse. And prompts and responses are still processed with zero data retention when structured outputs are enabled, which the runtime boundary depends on.

The investigation runs on the Agent SDK

The runtime decision came down to where session state lives. Managed Agents, in beta as of this writing, fits a long investigation well: durable sessions, mid-run steering, and scheduled runs, and moving there from an SDK prototype is a common transition path. We did not start there because Managed Agents keeps the session, the conversation history and its outputs, on Anthropic's infrastructure. The self-hosted sandbox release in May moved tool execution inside the customer perimeter; the loop and its transcript stay hosted. That design is what makes those features possible, and it is also why the hosted form is not eligible for zero data retention. For a bank, that constraint decidied the initial implementation design.



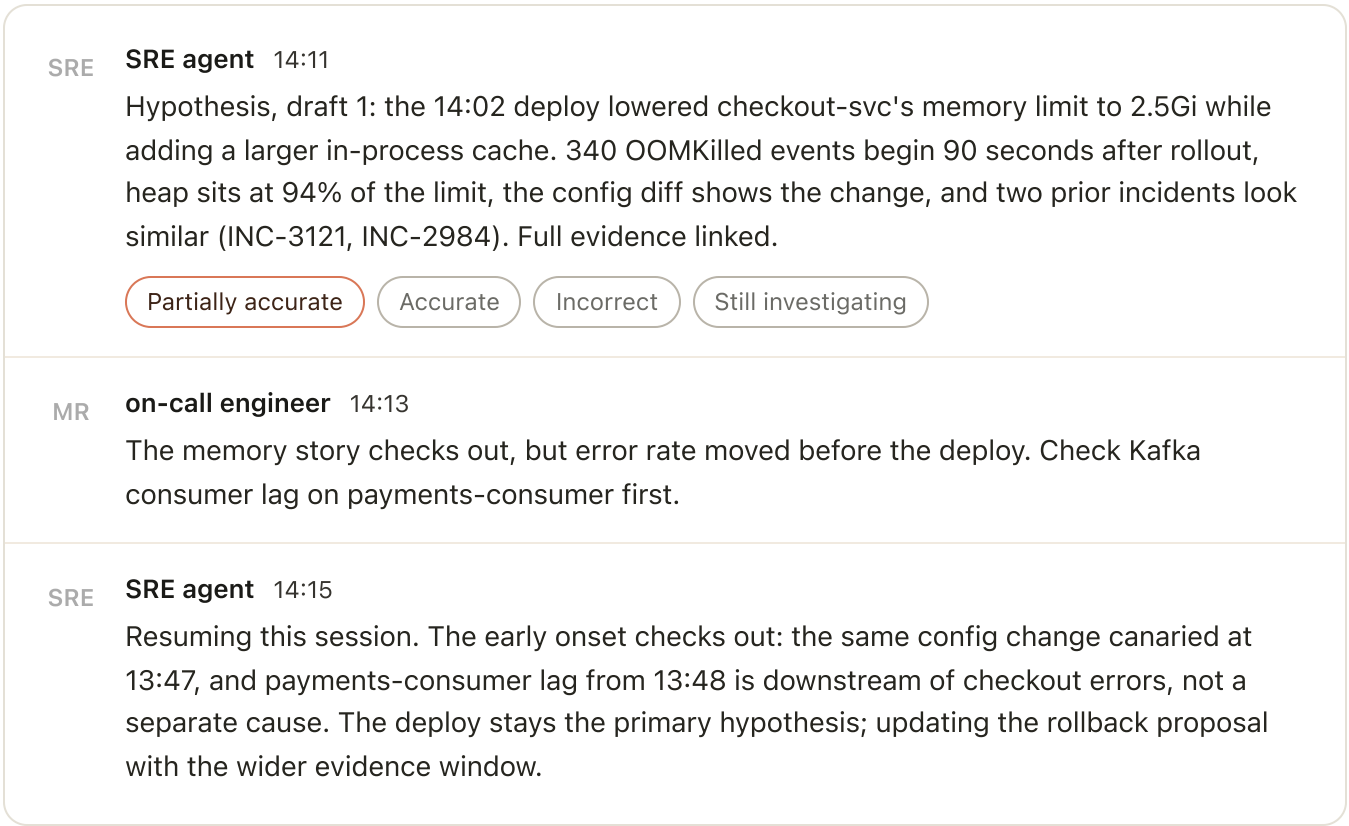
Only inference crosses the wire. Harness, tools, sessions, secrets, and authority stay inside.

Prompts and completions are the only traffic that crosses the boundary. Session state, tools, secrets, and write authority stay on bank infrastructure. Managed Agents with a self-hosted sandbox is the next progression of the design.

The Agent SDK provides the same loop, tools, and context management that power Claude Code, running in our process, with session state as JSONL files we hold. Skills load from the filesystem, MCP servers attach as configuration, and hooks let us enforce policy in code around every tool call. A simplified version of the investigator setup:

```
from claude_agent_sdk import query, ClaudeAgentOptions, HookMatcher

async def enforce_read_only(input_data, tool_use_id, context):
    # Writes are only legal through the action gateway, which is not
    # in this session's toolset. Block anything that could mutate
    # state, and log every attempt.
    tool = input_data.get("tool_name", "")
    if tool in {"Write", "Edit", "Bash"}:
        return {"hookSpecificOutput": {
            "hookEventName": "PreToolUse",
            "permissionDecision": "deny",
            "permissionDecisionReason": "writes go through the action gateway",
        }}
    audit(tool_use_id, input_data)
    return {}

options = ClaudeAgentOptions(
    system_prompt=INVESTIGATOR_PROMPT,
    setting_sources=["project"],  # loads .claude/skills: global, domain, service
    mcp_servers={"obs": OBS_SERVER, "incidents": INDEX_SERVER,
                 "sandbox": SANDBOX_SERVER},
    allowed_tools=[
        "Read", "Grep", "Skill",  # the Skill tool activates the runbook catalog
        "mcp__obs__query_metrics",
        "mcp__obs__search_logs",
        "mcp__obs__get_recent_deploys",
        "mcp__incidents__search_similar",
        "mcp__sandbox__run_analysis",
    ],
    hooks={"PreToolUse": [HookMatcher(matcher=".*", hooks=[enforce_read_only])]},
)

async for message in query(prompt=envelope_json, options=options):
    handle(message)
```

Steering needs no extra machinery, because sessions are resumable. The first message carries a `session_id`, and a follow-up from the incident thread resumes it with `ClaudeAgentOptions(resume=session_id)`. The engineer redirects the investigation instead of restarting it, and the session keeps everything it has already read and concluded. It is also why the investigation is one session rather than a fan-out of subagents: a single accountable hypothesis thread, steerable in one place, with parallelism pushed down into the analysis code. In the thread, that looks like this:



The steering loop, shown on an illustrative incident.

Sessions that run for hours eventually compact. It is essential that the following survive beyond summarization: the incident envelope, confirmed findings, open hypotheses, and action state live as files in the session workspace, written by the harness from the structured records the agent emits rather than by the agent itself. These are re-read on demand so that compaction can drop transcript without dropping the case. Traces mark any conclusion reached after a compaction, and reviewers weigh those accordingly. Cross-incident recall follows the same rule: the platform's memory tool would give the agent persistent files of its own, but recall at a

bank has to be shared across responders, audited, and governed on our side of the boundary, so it lives in the incident index as retrieval instead.

Those same files also handle escalation, cheap to detect and rare to pay for. An investigation stalls when the session exhausts its tool-call and thinking budget without producing a corroborated hypothesis, meaning one supported by the deploy diff, the metrics, and the logs together. The harness detects this mechanically: the structured record it maintains has a dedicated field for that hypothesis, so when the budget runs out, the field is either filled or the session has stalled.

On a stall, the harness escalates to an Opus-class consult. The consult receives the case file but not the transcript, which at that point is mostly a record of failed attempts. The consult stays advisory. It returns a structured advice record specifying which reads to run next and which hypotheses to prioritize, and that record is appended to the session for the Sonnet-class investigator to act on when it resumes.

In practice, about one investigation in twenty stalls. Roughly two-thirds of those reach a corroborated hypothesis after the consult; the rest are handed to the on-call engineer with the case file attached, which is where they were headed anyway. And because Opus runs only on stalled sessions, the typical investigation never pays for it: the consult's cost sits in the tail of the distribution.

Keeping evidence out of the context window

An investigation touches a lot of data the model should never hold. Six hours of error logs might be tens of thousands of lines; the useful signal is that 340 of them are OOM kills and the first one landed 90 seconds after a deploy. Context is for reasoning: fetching, filtering, and counting happen in code next to the data, and only conclusions come back. Claude writes a small program that calls the read tools, filters and joins the results, and returns one compact evidence object, and only that object enters the context window. On the Messages API, the managed version of the pattern is programmatic tool calling, enabled by adding `allowed_callers` to a tool definition alongside the code execution tool:

```
tools=[
    {"type": "code_execution_20260120", "name": "code_execution"},
    {
        "name": "search_logs",
        "description": "Search service logs. Returns JSON rows with "
                       "fields: ts, level, message, pod.",
        "input_schema": {
            "type": "object",
            "properties": {
                "service": {"type": "string"},
                "level": {"type": "string"},
                "hours": {"type": "integer"},
            },
            "required": ["service", "hours"],
        },
        "allowed_callers": ["code_execution_20260120"],
    },
    # get_recent_deploys and query_metrics are defined the same way
]
```

We do not run our telemetry through the hosted pass, for two reasons. One, MCP-connector tools cannot be called programmatically, which is why the snippet above defines the reads as plain custom tools; the limit is the connector's, and the MCP servers the Agent SDK attaches are unaffected. Two, the managed container runs on Anthropic infrastructure and retains execution artifacts for up to 30 days, the same class of constraint that kept sessions in-house. So the investigator runs the identical pattern behind one bank-side tool. `run_analysis` takes the program Claude writes, executes it in a network-restricted sandbox next to the read APIs, and returns only what it prints; the docs describe this self-managed sandbox alongside the hosted one, and for this data it is the only one we can use. Claude's side is unchanged either way:

```
# code Claude writes; executes in the bank-side sandbox
import json

deploys = json.loads(await get_recent_deploys({"service": "checkout", "hours": 6}))
logs = await search_logs({"service": "checkout", "level": "ERROR", "hours": 6})
oom = [line for line in logs.splitlines() if "OOMKilled" in line]

print(json.dumps({
    "deploy_ids": [d["id"] for d in deploys],
```

```
    "oom_count": len(oom),
    "first_oom_ts": oom[0].split()[0] if oom else None,
}))
```

We reserve the pattern for the read-heavy phase and keep the triage fast path on direct calls, since programmatic calling adds overhead when a turn makes only one or two calls. Anthropic's published benchmark shows the same split: a token reduction on a large read-heavy tool agent, a small penalty on low-call workloads.

Tool Search handles the adjacent problem of a catalog too large to load up front: deferred tools (the `defer_loading` mechanism) stay out of context until Claude searches for them, so a session opens with the incident envelope, the safety instructions, and a handful of common tools rather than every connector we operate.

The same discipline is the cost model. Prompt caching holds the stable prefix, the system prompt, tool definitions, and Skill metadata, so an hours-long session pays for its methodology once. Triage runs on Haiku, the investigator on a Sonnet-class model, and because only evidence objects enter context, token spend scales with findings rather than with log volume. Reasoning is the third axis of the same discipline: triage runs with extended thinking off, which is much of why it is cheap, while the investigator carries a thinking budget sized to the case. Calls, models, and thinking depth are right-sized together.

Writes go the other way. Rather than letting the model chain five small mutations, each write is a single tool, `rollback_deployment` for example, that owns its preconditions, dry run, approval routing, idempotency key, and inverse action.

The programmatic tool calling documentation is explicit that `allowed_callers` guides the model but is not a hard block, and should not be relied on as a security boundary. We take the same position one level up: no tool annotation, prompt, or Skill grants write authority.

The gateway checks every write independently, which is also what keeps the system safe against prompt injection arriving through log lines and tickets. A tool result can inform a hypothesis; it cannot grant permission. The residual risk sits one layer up: injected text cannot approve anything, but it can steer a wrong diagnosis into a

plausible proposal for a tired human at 3 a.m. Two habits blunt it. Every claim in a proposal carries its source link, and no mitigation is proposed on the testimony of a single source.

Runbooks as Skills

The knowledge that makes an investigation good is owned by service teams, not by us, so it ships as Skills in their repositories rather than as sections of a central prompt. A Skill is a folder with a `SKILL.md`. The frontmatter is what Claude matches a task against, and the body loads only when it matches:

```
---
name: service-checkout
description: Investigate checkout-svc incidents. Use when an alert names
  checkout-svc or symptoms include payment failures at checkout.
---

# What this service does
# Failure signatures
# Key dashboards and queries
# Dependencies and ownership
# Safe mitigations
# Escalate or prohibit
```

Skills compose by specificity. A checkout OOM session loads the global incident discipline, the OOM domain playbook, and the checkout service Skill. The global Skill says to diagnose before proposing a change. The domain Skill explains how to tell a lowered memory limit from a leak. The service Skill knows checkout runs on cost-constrained nodes, so the obvious fix, raising the limit, is the wrong one there. Until a Skill matches, it costs only its metadata in context, the name and description above, which is what lets the catalog grow to hundreds of runbooks without inflating every session.

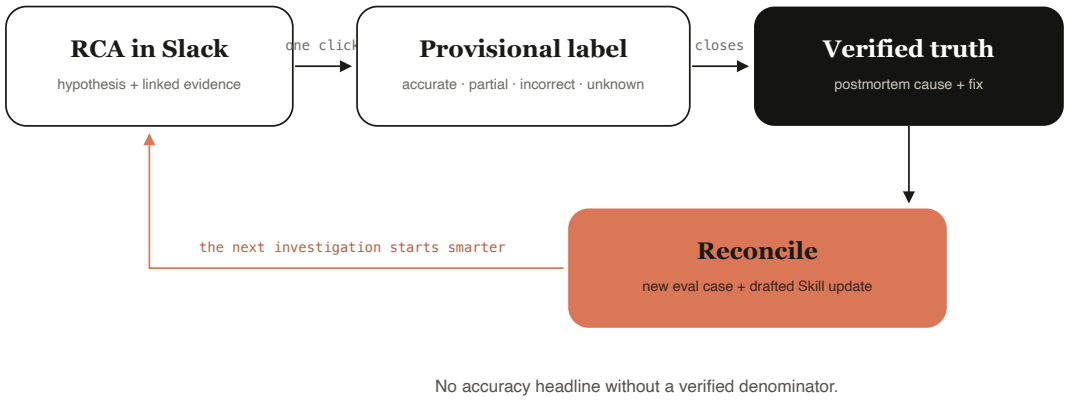
A service does not need a Skill to onboard; it starts on the global and domain tiers and gets its own the first time an incident teaches it something. Teams write and ship these without the platform team in the loop. `sre skill from-incident INC-4821`

scaffolds a draft from a closed postmortem, local replay through the Agent SDK runs it against the service's recorded incidents, and CI will not publish a Skill without a named owner and a passing eval set. That division of labor is what lets the catalog scale: we run the tooling, service teams own what is true about their services, and the signal we watch is whether a team's second Skill ships without anyone from our team involved.

Policy Skills are the exception. They describe rules like data-access boundaries and change freezes so the model can reason about them, but they enforce nothing. Enforcement stays in the hook and the gateway. A Skill can make the agent smarter about policy; it cannot authorize an action.

Measuring whether a diagnosis was right

A root-cause hypothesis does not score itself. A fluent explanation can be wrong, and at the moment the agent posts it, the true cause is often unknown to the humans too. So each diagnosis gets two labels at two different times.



Engineers label the diagnosis in the incident thread while it is live. After the review closes, the verified cause is reconciled against that label, and the pair becomes an eval case plus a drafted Skill update for the owning team.

The provisional label is a one-click verdict in the incident thread, accurate, partially accurate, incorrect, or still investigating, with an optional correction. The verified

label comes later, reconciled against the post-incident review's confirmed cause and fix. Every reconciled pair becomes an eval case, and the replay path depends on the artifact. Bounded calls, triage envelopes and judge scoring, rerun through Message Batches at half the interactive price. Full investigations replay through the Agent SDK against recorded tool results: every MCP read a live session makes is captured as a fixture, so a closed incident becomes a reproducible case a Skill change can be tested against before it ships. When a new trajectory requests a read the fixtures do not hold, the case fails loudly rather than falling through to live systems. The capture layer is the quiet workhorse here; recording reads during live incidents is what turns history into tests. A replay is scored in layers, cheap checks first: the record validates against its schema, every read it claims exists in the fixtures, and no denied tool was attempted. The judge runs last because it is the expensive opinion.

Two reporting rules keep the numbers honest. We never merge unknown into incorrect, and we never compute accuracy over incidents that lack a verified final cause. On business impact, time-to-hypothesis is the metric the agent directly moves. MTTR stays the north star, but most of an incident's wall clock is waiting, coordination, and recovery rather than diagnosis, so we claim an MTTR effect only from a controlled comparison, not from a before-and-after.

Checking the work before it posts

The corroboration rule, that the deploy diff, the metrics, and the logs have to agree before anything posts, started as a sentence in the system prompt and a habit of whoever read the thread. The rule exists because the cheapest way for an investigator to be wrong is to fixate on the first plausible cause. It is now a mechanism. Before a diagnosis posts to Slack or a proposal reaches the gateway, a grading pass, a single Haiku-class structured call, re-reads the record, follows each claim's source link, and checks that the excerpt supports the claim. A failure bounces the record back into the session with the failing claim named; the bounce is just a resume, the same mechanism a human steer uses, and it is capped at two, after which the diagnosis posts flagged instead of clean rather than being suppressed, because a wrong but visible hypothesis in the thread is recoverable and a silently withheld one is not.

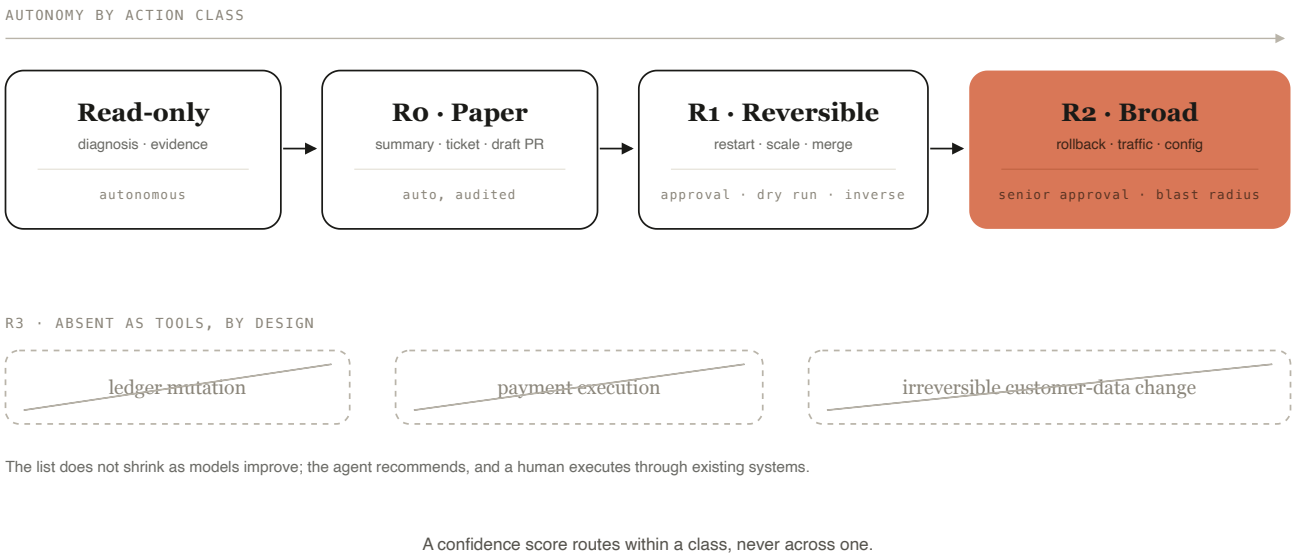
A bounce is itself a typed record:

```
{
  "record": "dx_4821",
  "verdict": "bounce",
  "failing_claim": "onset precedes the 14:02 deploy",
  "check": "cited excerpt starts at 14:02; the claim needs the 13:47 canary window",
  "bounce_count": 1,
  "disposition": "resume, failing claim named"
}
```

Two properties keep the grader inside the trust model. It can block and bounce; it cannot approve, so it adds no authority anywhere. And its verdicts land in the eval store like any other artifact and reconcile against verified causes like any other label, because a checker with an unmeasured error rate is one more opinion. In a domain where being wrong has consequences, the verification logic between the model and execution is the part of the harness most worth owning; the grader is that logic promoted from a rule to a job, at about two cents a pass.

Gating actions

Reads are autonomous where data policy permits. Writes are transactions, and each action class has its own promotion requirements:



Autonomy widens down the ladder only when a class's evidence supports it. R3 sits apart because its tools were never built.

R0 covers paper actions, posting a summary, updating a ticket, opening a draft PR, which run automatically with an audit trail because they do not touch production state. R1 covers bounded reversible writes behind approval, a dry run, and a tested inverse. R2 covers wider changes like rollbacks and traffic shifts, which add senior approval and an explicit blast-radius report. R3, anything irreversible or money-affecting, is handled by not building the tool. The agent can recommend a ledger correction; a human executes it through existing systems. Promotion between tiers happens per action class and requires evidence: enough verified cases, eval thresholds met, shadow-mode performance, and low override rates. A high confidence score routes a case within its tier and never moves it across one.

The proposal that reaches the gateway is itself a structured output, and strict tool use holds the write tools' inputs to their schemas the same way, so the record is typed end to end. The gateway takes typed records and nothing else, which is what keeps approval, replay, and audit uniform across action classes:

```
{
  "proposal_id": "prop_01j9k2",
  "incident": "INC-4821",
  "action": "rollback_deployment",
  "target": {"service": "checkout-svc",
```

```
              "from": "2026.07.14-3", "to": "2026.07.14-2"},
  "risk_tier": "R2",
  "preconditions": {"deploy_correlated": true,
                    "change_freeze": false,
                    "inverse_tested": true},
  "dry_run": {"status": "passed",
              "blast_radius": "1 service, 12 pods, no dependent config"},
  "idempotency_key": "inc-4821-rollback-1",
  "approval": {"route": "senior-oncall", "state": "pending"},
  "expires_at": "2026-07-19T14:40:00Z"
}
```

That record was generated in shadow, because R2 still runs there: every R2 proposal is evaluated end to end, dry run included, while execution is withheld. R1 is past that stage; restart, scale, and merge execute in production behind approval, each with its dry run and tested inverse.

Diagnosis as a capability

The investigator's surface was already a function: a typed envelope in, a typed and sourced diagnosis out, nothing but reads in between. In the spring we wrapped that function in an MCP server, which turns diagnosis into a capability other agents call rather than a product only alerts can start. The deploy pipeline's agent asks whether its rollout caused the errors it is watching before deciding to proceed; the capacity planner asks for six hours of history on a service before recommending a scale-down; the next internal assistant gets root-cause analysis without rebuilding it.

Nothing in the trust model moves. The caller receives a hypothesis and its evidence, never authority; a proposal still exits only through the gateway; and agent-called sessions are admitted below alert-triggered ones, so a chatty caller cannot starve an incident. One thing does move, and it is the measurement contract: an investigation was defined as a session triggered by a real alert, so agent-called sessions are a third category next to live and shadow, reported separately, and a colleague agent's curiosity never inflates the incident numbers.

When the assistant is down

An AI SRE has to plan for its own absence, and the correlation is uncomfortable: the assistant is most likely to be impaired during the largest incidents, when regional trouble degrades the same networks and providers everything else depends on. So it is additive by construction. The incident process ran for years without it and still can. If the envelope call misses its timeout budget after one retry, the alert routes exactly as it did before. If a session dies mid-investigation, the thread says so and the on-call keeps working. A circuit breaker on the API path drops the whole system to the human-only flow and posts one line saying it did. The fast path and the investigator fail independently, and nothing in paging, severity, or escalation blocks on either. The residual exposure is capacity at the provider's end, and part of that is bought rather than engineered: the system runs on committed Priority Tier capacity, which targets 99.5 percent uptime with prioritized compute and falls back to the standard tier past the commitment.

Alert storms get the same treatment. A regional event can fire hundreds of alerts in minutes, and starting hundreds of investigations would be an incident of its own. Envelopes deduplicate on service and error signature into one investigation with a fan-in list, concurrent sessions are capped and admitted by severity, and separate workspaces with independent rate limits keep triage from starving behind investigation traffic.

Keeping it current

The failure mode we planned for first is stale knowledge, and the mechanism against it is to attach freshness to work that already happens. When a post-incident review closes, the incident index updates, an eval case is created, and a single Messages API call drafts the corresponding Skill update with links back to the postmortem. The draft goes to the owning team as a pull request and never publishes on its own, because postmortems can be incomplete or wrong too. Skill changes trigger targeted evals in CI through change-impact mapping, so a checkout change reruns the checkout and OOM cases rather than the entire corpus, and the full suite runs on a schedule and on higher-risk changes.

Model upgrades get the same treatment as any other risky change. We rerun the full eval suite, compare tool trajectories and costs, and shadow before promoting, because a newer model can be better on average while quietly changing a behavior a workflow depended on. Treating a model swap as a migration rather than a drop-in matters because the failure is quiet: a headline metric can hold while the tool trajectory shifts underneath it, so the diff we review is trajectories and costs, not accuracy alone.

The harness is tuned to one model family at a time rather than routed across several, which is why a swap is a migration and not a config change, and the migration diff asks a second question now: not just what changed but what can be removed. Early harness text was scaffolding, walls that made a weaker model walk straight, and a newer model consumes that scaffolding, so the candidates are the steering parts: prompt passages that correct a previous model's habits, exemplars demonstrating a format structured outputs already guarantees. In the last migration the eval ran twice, once with the harness as is and once without three such candidates, including the sentence telling the model not to write, and the smaller harness matched, so it promoted. Deleting that sentence was safe precisely because the sentence was scaffolding and the hook is authority: the hook denies writes either way, and the denial counter, flat through the change, priced the deletion. Authority is never a candidate. A harness that only ever grows is accumulating debt against models that no longer need it.

The measurement contract

The numbers this system produces will publish later than the design did, which creates a temptation this section exists to remove: the temptation to shape the figures to the story once they arrive. So the contract comes first. Before any inventory publishes, here is what the words will mean. An investigation is a session triggered by a real alert; replay executions in CI are never counted, however many times the fixtures rerun. Live means the diagnosis posted into an incident thread where an engineer could label or redirect it; shadow means it ran without posting. A session started by another agent through the diagnosis surface is agent-called; it is reported on its own and kept out of the live and shadow counts. A steer is a substantive human redirect, not a bare resume and not a label click, both of which also resume the

session. A grader bounce resumes the session too, machine-initiated, and counts as neither a steer nor a label. The eval corpus is reported as the curated, deduplicated set a change is actually tested against, with raw fixture capture described separately, so a corpus count never masquerades as a capture rate. Reporting windows are trailing 90 days, with cumulative totals only where a store genuinely accumulates.

And here is what any published figures must satisfy, checkable by a reader. Reconciled pairs are fewer than labeled diagnoses, which are fewer than live investigations. Dry runs equal proposals. Executions never exceed proposals, flagged posts never exceed grader bounces, and R2 executions are zero until R2 is promoted, which has not happened and has no date.

The label rate publishes; the label distribution does not, because the distribution reconstructs the accuracy figure we refuse to compute over an unverified denominator. The reconciliation crosstab publishes as a direction and a design response, never as percentages, for the same reason. Every figure ships rounded, dated, with its definition attached and its query retained. A number that cannot meet these terms is not published late; it is not published.

Where it stands

As of July 2026, the rollout is deliberately sequenced: the v2 rolled out in phases behind the beta, earliest pieces first. The fast path and the investigator came up before any write capability. The action gateway is live for R0 paper actions and, having cleared its shadow period, for R1 reversible writes behind approval; R2 remains in shadow with no promotion date, deliberately. The eval library grows with every closed review.

The boring inventory, under the contract above. Over the trailing 90 days the agent ran roughly 96,000 investigations on real alerts, about 71,000 live in incident threads and 25,000 in shadow, across 240 of 264 tier-1 services, with about 55 tier-2 services in early rollout; the cumulative total since rollout began is around one million. About 12,000 live sessions, one in six, were steered mid-investigation. About 4,800 investigations stalled into an Opus consult, and about 3,200 of those reached a

corroborated hypothesis on the resume. Since the diagnosis surface opened in the spring, other agents have started roughly 1,900 agent-called sessions, which the contract reports outside the investigation counts here.

The registry holds about 760 Skills, 6 global, 34 domain, and roughly 720 service, with 86 percent of the service Skills carrying no platform-team commits, and about 180 teams have shipped a second Skill with no platform-team commits or reviews, the signal we actually watch.

The grader bounced roughly 6,900 diagnoses at least once before they posted, and about 900 posted flagged after a second failed pass. 64 percent of live diagnoses received a one-click label. The eval store holds about 9,400 curated fixture-backed incidents and 5,200 reconciled pairs, both cumulative, with raw capture running far larger and continuous. Over that verified set, the diagnosis matched the postmortem's confirmed cause 96 percent of the time.

On the action side, about 168,000 R0 paper actions executed over the window, mostly summaries and ticket updates plus around 12,000 draft pull requests. Roughly 4,900 R1 proposals were generated, every one dry-run, and about 3,400 executed in production behind approval; the rest were rejected or expired at the gate, which is the gate doing its job. About 1,300 R2 proposals ran end to end in shadow, dry runs matching proposals one for one, with zero executions. The read-only hook denied and logged about 2,600 write attempts, most of them the model reaching for scratch files or a shell mid-investigation, and about 180 of them attempts to apply a fix directly rather than propose it.

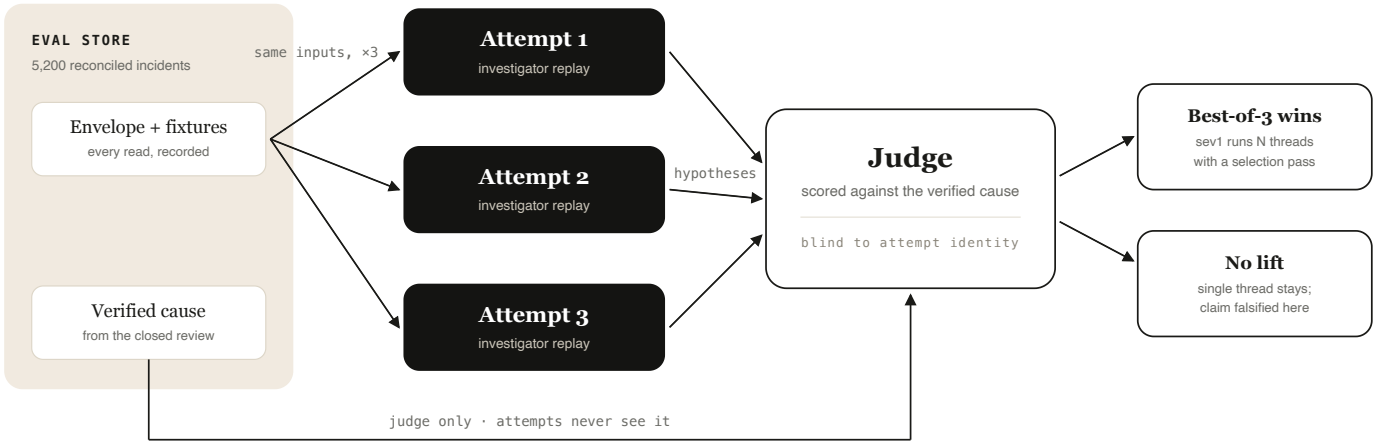
The median investigation costs about 70 cents, the ninetieth percentile about $3.20 for the long ones, and triage stays under two cents on Haiku. The circuit breaker has dropped the system to the human-only flow 14 times since rollout, twice in the window, and about 2,300 alert storms collapsed into single investigations, the largest folding roughly 4,800 alerts into one with a fan-in list.

Two numbers we do not report: an MTTR improvement, because we do not yet have a controlled comparison, and any accuracy figure over incidents without a verified cause.

Where it goes next

Step back from the components and the system is a set of jobs. One classifies, one investigates, one checks the work before it posts, one advises a stalled session, one scores replays, and one writes what an incident taught back into the Skill catalog through pull requests. The jobs share a shape: a narrow responsibility, a typed record as output, and no authority beyond it. That is the conceptual direction of this design, an agent system that improves by splitting work into jobs it can measure separately, and it is why diagnosis itself became a job other agents can call. One job is missing. Nothing here ever runs the same investigation twice and picks the better answer.

Once an agent works, teams believe they hold two levers, a bigger model or a longer run. A third exists in principle: run N attempts and select among them. The claim that it returns real accuracy goes untested in production systems, because productionizing the selection is genuinely hard, so almost nobody has a number for it on their own task. That last clause is the part this system can do something about, because we do not have to productionize anything to get the number. The eval store holds 5,200 reconciled incidents whose fixtures answer every read a replay makes and whose true cause is verified. Replaying a stratified sample three ways through the Agent SDK, same envelope, same fixtures, independent sessions, and judge-scoring each attempt's final hypothesis against the verified cause says whether best-of-3 beats a single run on incident diagnosis, before a production token is spent. Three runs across a 300-incident sample is roughly six hundred dollars of tokens at the median investigation cost. The attempts never see the verified cause; only the judge does, and it scores blind to which attempt produced which hypothesis.



Fixtures turn the experiment into replays, and the protocol publishes with the result.

A stratified sample of reconciled incidents replays three ways against recorded fixtures. The judge alone sees the verified cause and scores blind to attempt identity, and the adoption rule is written before the first replay.

The measurement contract's habit applies to experiments too, so the protocol is fixed before the first replay:

```
# fixed before the first replay runs
sample        300 reconciled incidents, stratified by severity and tier
attempts      3 per incident; independent sessions; identical inputs
judge         corroboration rubric, scored against the verified cause,
              blind to which attempt produced which hypothesis
comparison    judge-selected best-of-3 vs. attempt 1 alone
adopt when    sev1 accuracy improves by 2+ points and the two added
              attempts stay under 5% of fleet investigation spend
either way    the result publishes with this protocol attached
```

The 96 percent headline leaves little room, which is itself the point: if a third lever exists, the only place it can show up is the slice where single runs miss, and sev1 is the only slice where parallel sessions could ever be worth their cost. The cost bar is a spend ceiling rather than a per-diagnosis ratio deliberately: tripling attempts triples cost by construction, so a cost-per-correct bar could never pass and a bar that cannot pass is theater, while sev1's small share of volume is what makes the ceiling cheap to stay under. If the number clears the bar, sev1 investigations get N hypothesis threads

and a selection pass, and disagreement between threads surfaces in the incident thread rather than being resolved silently, because two independent sessions reaching different causes from the same evidence is information the on-call should see, not noise for a judge to hide. If it does not clear, we have falsified the claim on our own task for the cost of a replay, and the single thread stays. Either way, the claim made earlier, that an investigation is one session rather than a fan-out of subagents, gets tested instead of defended. Best-of-N is not that fan-out; each attempt is still a single accountable thread, and the judge is a separate job, not a coordinator inside the session. The sentence stands today. If the experiment wins, it gains a qualifier, one accountable thread per hypothesis, and the qualifier will arrive with the experiment's number attached.

The conditional box in the boundary figure, meanwhile, aged in a specific direction. The May release that moved tool execution inside the perimeter also shipped MCP tunnels, in research preview, which reach MCP servers inside a private network without exposing them; the sandboxes run in public beta with Cloudflare, Daytona, Modal, and Vercel, or on infrastructure the customer controls. That covers execution and tool connectivity, two of the three things our boundary keeps inside. The session still runs hosted, so the condition on the box has narrowed to exactly the term it was drawn around: where the session lives. When that fits, the migration is small: compaction, caching, resume, and scheduling are native on the hosted runtime, so the loop supervisor and the scheduler get deleted, and the gateway, the incident index, the eval store, and the Skills registry stay where they are, because they were never the runtime's to hold. The swap validates the way a model swap does: the fixture corpus replays on both runtimes, and the hosted form promotes when it matches.

Takeaways

1. Choose the runtime by deployment constraint first, capability second. For us, session-state residency settled the question before feature comparison started, which put the investigation on the Agent SDK, with Managed Agents as the hosted form of the same design.
2. Right-size every call. Bounded tasks run as single Messages API calls with structured outputs; only the open-ended investigation pays for an agent session,

and the largest model prices into the stall path, not the median.

3. Keep raw telemetry out of the context window. Code in the middle does it, through programmatic tool calling where retention terms fit and a bank-side sandbox where they don't; Tool Search keeps the catalog out of context, and evidence objects instead of dumps do it on the tool side.
4. Put procedures in Skills and authority in code. A Skill, a prompt, or a tool annotation can make the agent smarter; none of them can authorize an action.
5. A checker can block but never approve. The grader bounces unsupported claims back into the session, posts flagged rather than suppressing, and its verdicts reconcile like any other label, so the checker has an error rate instead of a reputation.
6. Label every diagnosis twice, once in the moment and once against the verified postmortem, and only compute accuracy over the verified set.
7. The Skill catalog scales through ownership. Postmortems draft the updates, CI gates the publishing, and service teams, not a central team, keep the content true.
8. Test strategy claims on the corpus before adopting them. A replayable, verified incident set turns a roadmap debate, best-of-N included, into a pre-registered experiment that costs a replay rather than a rollout.

Referenced sources and further reading

- Scaling Managed Agents: Decoupling the brain from the hands
- Advanced tool use and the programmatic tool calling reference
- Writing effective tools for agents
- Code execution with MCP
- Claude Agent SDK overview
- Structured outputs
- The SRE incident responder in the Managed Agents cookbook and the site reliability agent in the Agent SDK series
- Agent Skills overview
- Demystifying evals for AI agents

- Building an Ecosystem, not a Walled Garden, Katelyn Lesse and Angela Jiang on Sequoia's Training Data
- Self-hosted sandboxes and MCP tunnels in Managed Agents

Ada, the read-only beta, predates this work and proved the reasoning use case. The architecture here is the v2 design and phased implementation I led, with incident-response, observability, security, and service teams owning their parts. Platform statuses and API syntax checked against Anthropic documentation, July 2026.